

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ «ЛЭТИ» ИМ.В.И.УЛЬЯНОВА (ЛЕНИНА)»**

**ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ И ИНФОРМАТИКИ
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ**

**Курсовая работа
по дисциплине «Алгоритмы и структуры данных»
на тему «Графы»**

Выполнили студенты группы 3312

Лебедев И.А.

Шарапов И.Д.

Принял старший преподаватель Колинько П.Г.

Санкт-Петербург
2024

Содержание

Текст индивидуального задания	3
Математическая формулировка задачи в терминах теории множеств.....	3
Выбор и обоснование способа представления данных	4
Описание алгоритма и оценка его временной сложности	4
Набор тестов и результаты проверки алгоритма на ЭВМ	5
Выводы	8
Список использованных источников	9
Приложение	10

Текст индивидуального задания

Получить множество двудольных компонент неориентированного графа. Удалить рёбра исходного графа так, чтобы не разрушилась ни одна компонента связности и граф стал двудольным.

Входные данные: В первой строке вводится 2 числа n и m . Далее в m строках вводится по 2 числа a и b (ребро между вершинами с номером a и b).

Выходные данные: Вывести все рёбра полученного графа.

Математическая формулировка задачи в терминах теории множеств

Пусть задан неориентированный граф $G = (V, E)$, где V — множество вершин, а E — множество рёбер. Граф G называется *двудольным*, если существует разбиение множества вершин V на два непересекающихся подмножества V_1 и V_2 такие, что $V = V_1 \cup V_2, V_1 \cap V_2 = \emptyset$, и для всех рёбер $(u, v) \in E$ выполнено: $u \in V_1 \Rightarrow v \in V_2, v \in V_1 \Rightarrow u \in V_2$. Иными словами, никакое ребро не соединяет две вершины внутри одного и того же подмножества V_1 или V_2 .

Пусть граф G состоит из k компонент связности $G_1, G_2, \dots, G_k$. Каждая компонента $G_i = (V_i, E_i)$ является подграфом G со своим множеством вершин V_i и рёбер E_i . Если для некоторой компоненты G_i существует разбиение $V_i = V_{i,1} \cup V_{i,2}$ с описанным выше свойством, то G_i называется *двудольной компонентой* графа G .

Чтобы выделить двудольные компоненты графа G , мы можем отыскать внутри каждой компоненты остовное дерево. Рассмотрим дерево $T = (V_T, E_T)$ и выберем в нём произвольную вершину $v_0 \in V_T$ в качестве корня. Пусть $v_0 \in V_1$. Все вершины v_i , такие что $d(v_0, v_i)$ — чётное, также лежат в V_1 . Оставшиеся вершины ($d(v_0, v_i)$ — нечётное) лежат в V_2 . Так как переход по ребру меняет чётность расстояния до корня, никакое ребро не соединяет вершины внутри одного и того же множества V_1 или V_2 . Таким образом, (V_1, V_2) — это двудольное разбиение V_T . Значит любое дерево является двудольным графом.

Объединяя найденные остовные деревья для всех связных компонент, мы формируем подграф $G' = (V, E')$, где каждое множество рёбер $E'_i \subseteq E_i$ образует двудольную компоненту исходного графа G .

Выбор и обоснование способа представления данных

В памяти ЭВМ граф представляется в виде списка смежности. Это такой список, где каждая строчка соответствует вершине графа и содержит список всех соседних вершин. Преимущества списка смежности в нашей задаче:

- *Эффективен по памяти.* Список смежности использует $O(n + m)$ памяти. Каждый узел в списке хранит только своих соседей, что делает его компактным для разреженных графов. Матрица смежности требует $O(n^2)$ памяти, что становится проблемой при большом числе вершин.
- *Эффективен при обходе.* При BFS список смежности позволяет эффективно находить всех соседей текущей вершины. Список смежности позволяет непосредственно получать соседей текущей вершины без необходимости перебирать все вершины, как это было бы в матрице смежности.

Описание алгоритма и оценка его временной сложности

Исходный граф задаётся через список рёбер. После обработки программа оставляет только те рёбра, которые образуют остовные деревья в каждой компоненте связности. Таким образом, итоговый граф состоит из тех же компонент связности, что и исходный, но каждая из них становится деревом, а значит, автоматически является двудольной. Выделение деревьев, на наш взгляд, является наиболее наглядной формой для представления двудольных компонент.

Алгоритм начинается с чтения входных данных, где задаётся количество вершин n и рёбер m . Для представления графа используется список смежности, который создаётся за $O(m)$ времени, так как каждое ребро добавляется в список дважды (по одному разу для каждой вершины).

Затем для нахождения всех компонент связности графа выполняется обход в ширину (BFS) для каждой ещё не посещённой вершины. BFS проходит по всем вершинам и рёбрам текущей компоненты, добавляя рёбра, которые входят в остовное дерево. Каждая вершина посещается один раз, а каждое ребро обрабатывается ровно дважды. Таким образом, сложность обхода всех компонент связности составляет $O(n + m)$.

После завершения обхода компоненты связности подсчитываются, а остовные рёбра для каждой из них сохраняются в массиве *spanning_edges*. На последнем этапе программа выводит количество компонент связности и рёбра остовного дерева для каждой компоненты. Печать данных занимает $O(n)$, так как в каждой компоненте связности остаётся $n_i - 1$ ребро, где n_i — количество вершин в i -ой компоненте связности.

Итоговая временная сложность всего алгоритма составляет $O(n + m)$, где n — количество вершин, а m — количество рёбер. Эта сложность является оптимальной для задач обхода графов. Использование списка смежности обеспечивает компактное хранение данных и эффективный доступ к соседям каждой вершины, что делает алгоритм подходящим для разреженных графов.

Набор тестов и результаты проверки алгоритма на ЭВМ

Рассмотрим примеры запуска программы и визуализируем их. Жёлтым цветом выделен полученный граф, который остаётся после обработки исходного графа алгоритмом.

Ввод	Вывод
7 12 1 2 1 3 1 4 2 3 2 5 3 4 3 6 4 7 5 6 6 7 3 7 2 6	Count of connected components: 1 Component 1: 1-2 1-3 1-4 2-5 2-6 3-7

Таблица 1 – Пример №1

В первом примере показана корректная обработка графа с одной компонентой связности.

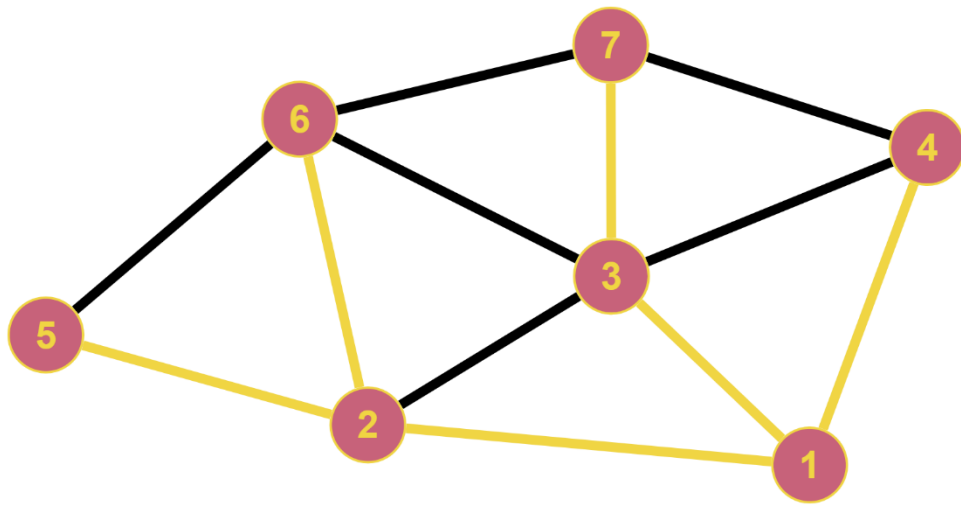


Рисунок 1 – Визуализация примера №1

Ввод	Вывод
10 11 3 4 2 3 1 4 1 3 1 2 5 7 5 6 6 7 8 10 9 10 8 9	Count of connected components: 3 Component 1: 1-4 1-3 1-2 Component 2: 5-7 5-6 Component 3: 8-10 8-9

Таблица 2 – Пример №2

Второй пример демонстрирует корректную работу алгоритма на графе с несколькими компонентами связности. При этом в каждой компоненте связности итогового графа рёбра идут от вершин с наименьшим номером. Это объясняется тем, что вершины перебираются в цикле от 1 до n .

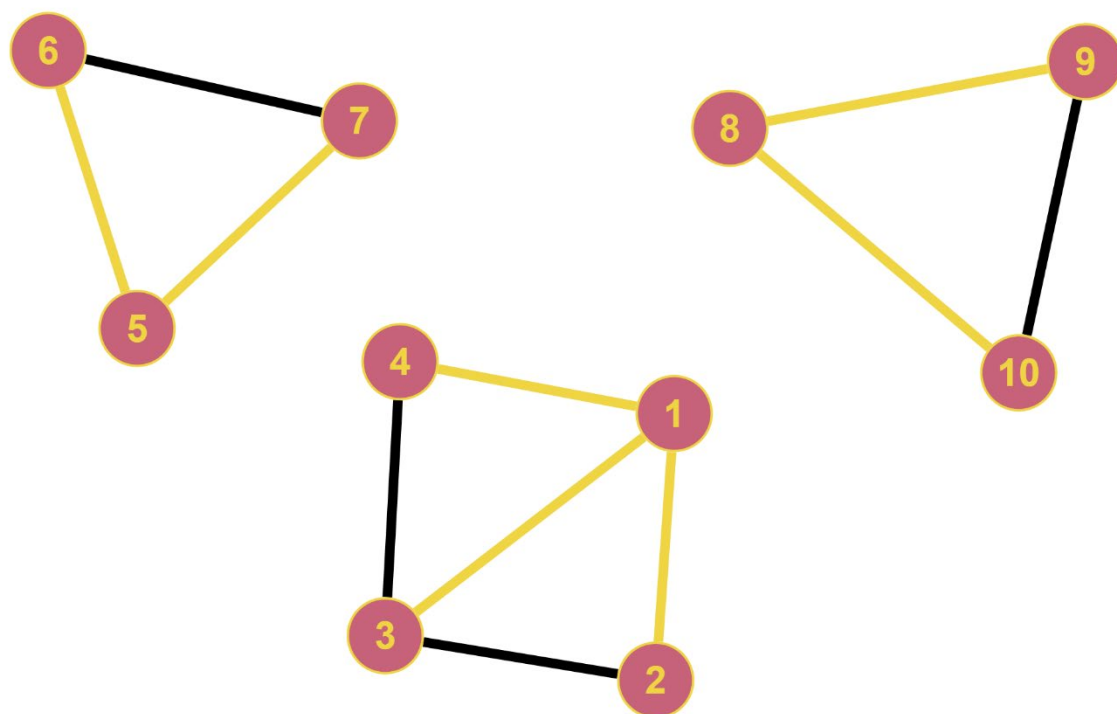


Рисунок 2 – Визуализация примера №2

Ввод	Вывод
15 40	Count of connected components: 2
1 5	Component 1:
2 4	1-5
3 6	1-7
1 7	1-2
6 7	1-4
4 7	1-6
1 2	1-3
5 6	
1 4	Component 2:
2 7	8-11
1 6	8-9
2 5	8-12
2 3	8-10
4 5	8-13
2 6	8-15
4 6	9-14
3 7	
3 5	
1 3	
5 7	
13 15	
12 14	
12 13	
9 15	
8 11	
8 9	

11 13	
13 14	
8 12	
8 10	
10 15	
9 14	
9 10	
8 13	
10 13	
8 15	
9 13	
9 11	
9 12	
10 14	

Таблица 3 – Пример №3

Третий пример показывает работу на плотном графе с большим количеством вершин.

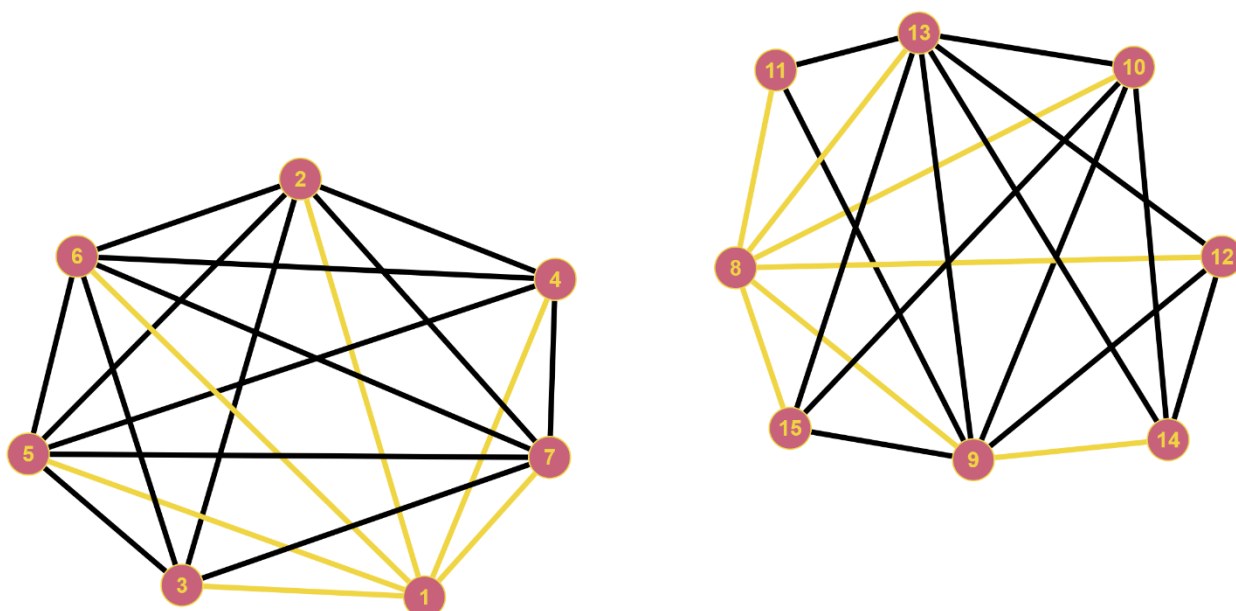


Рисунок 3 – Визуализация примера №3

Выводы

В ходе выполнения работы была решена задача выделения множества двудольных компонент из заданного неориентированного графа. Исходный граф был представлен в виде списка смежности, что позволило эффективно обрабатывать граф. Применённый алгоритм основан на методе обхода в ширину (BFS), с помощью которого были найдены все компоненты связности и построены их остовные деревья.

Основные результаты:

1. Разработанный алгоритм преобразует граф в подграф, сохраняя его компоненты связности и обеспечивая, что каждая из них становится двудольной. Это достигается за счёт удаления избыточных рёбер, оставляя только рёбра, образующие остовные деревья.

2. Реализация алгоритма имеет линейную временную сложность $O(n + m)$, что подтверждается теоретическими расчётами и результатами тестирования. Данная сложность является оптимальной для подобных задач.

3. Эффективность использования списка смежности подтверждена, так как эта структура данных позволяет экономить память и упрощает доступ к соседним вершинам, что критически важно при выполнении обхода графа.

4. Алгоритм успешно протестирован на различных типах графов: с одной компонентой связности, с несколькими компонентами, а также на плотных графах. Во всех случаях результаты работы программы корректны, что подтверждает её функциональность и надёжность.

Таким образом, поставленная цель достигнута: граф преобразован в множество двудольных компонент без нарушения связности, а разработанный алгоритм показал высокую эффективность и применимость для работы с разреженными графами.

Список использованных источников

1. Колинко П. Г. Пользовательские структуры данных: Методические указания по дисциплине «Алгоритмы и структуры данных, часть 1». — СПб.: СПбГЭТУ «ЛЭТИ», 2024. — 64 с. (вып.2408).

2. Поиск в ширину (BFS) [Электронный ресурс]. URL: <http://www.e-maxx-ru.lgb.ru/algo/bfs>.

Приложение

Код программы

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

int main() {
    int n, m, u, v;
    cin >> n >> m;
    vector<vector<int>>> g(n);
    for (int i = 0; i < m; ++i) {
        cin >> u >> v;
        u--;
        v--;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<bool> visited(n, false);
    int cnt = 0;
    vector<vector<pair<int, int>>> spanning_edges;
    queue<int> q;
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            spanning_edges.emplace_back();
            visited[i] = true;
            q.push(i);
            while (!q.empty()) {
                u = q.front();
                q.pop();
                for (auto w: g[u]) {
                    if (!visited[w]) {
                        visited[w] = true;
                        spanning_edges[cnt].emplace_back(u, w);
                        q.push(w);
                    }
                }
            }
            ++cnt;
        }
    }

    cout << "Count of connected components: " << cnt << "\n";
    for (int i = 0; i < cnt; ++i) {
        cout << "\nComponent " << i + 1 << ":\n";
        for (auto x : spanning_edges[i]) {
            cout << x.first + 1 << "-" << x.second + 1 << "\n";
        }
    }
    return 0;
}
```